

TreeMap Editor

Spécification Technique Complète

Version 2.0.0 · 30 mai 2026 · Angular 17 + Firebase

Propriété	Valeur
Projet	TreeMap Editor
Version	2.0.0
Date	30 mai 2026
Auteur	Équipe développement
Stack	Angular 17 · Firebase 12 · Electron 28 · TailwindCSS 3
Déploiement	Netlify (Web) · Electron (Desktop Windows)
Statut	Production

Ce document constitue la référence technique unique de l'application TreeMap Editor. Il décrit l'architecture logicielle, les modèles de données, les services, les composants, les flux d'authentification, la persistance des données et les considérations de déploiement.

Table des matières

§	Section	Page
1	Contexte et objectifs	3
2	Architecture générale	4
3	Stack technologique	5
4	Structure du projet	6
5	Modèles de données	7
6	Services Angular	9
7	Composants	12
8	Routage et guards	15
9	Authentification	16
10	Persistance et synchronisation	17
11	Gestion des thèmes	18
12	Export / Import	19
13	Mode impression PDF	19
14	Déploiement	20
15	Sécurité	21
16	Performances	22
17	Contraintes et limitations connues	22
18	Évolutions futures	23

1. Contexte et objectifs

1.1 Présentation du produit

TreeMap Editor est une application de cartographie hiérarchique (mind-map / tree-map) permettant à des équipes de créer, organiser et partager des structures arborescentes enrichies. Chaque nœud peut contenir du texte enrichi, des images, des listes de tâches (check-list) ou des blocs de code. Des métadonnées financières et temporelles (prix, jours, numéro de ticket) permettent de transformer l'outil en support de chiffrage ou de planning.

1.2 Objectifs fonctionnels

- Créer et gérer plusieurs TreeMaps (projets) depuis un tableau de bord centralisé.
- Éditer les nœuds de manière riche : titre, ticket, prix, jours, contenu multi-type.
- Organiser les nœuds par glisser-déposer ou via les contrôles de l'arbre.
- Synchroniser les données cross-device via Firebase Firestore (utilisateurs Google).
- Permettre un accès hors-ligne en mode invité avec persistance locale (localStorage).
- Exporter les TreeMaps en JSON et imprimer en PDF via le navigateur.
- Offrir trois thèmes visuels : Clair, Sombre, Contraste élevé.
- Fonctionner en application web (Netlify) et en application desktop (Electron/Windows).

1.3 Utilisateurs cibles

Profil	Besoin principal
Chef de projet	Chiffrage, planning, suivi tickets
Développeur	Documentation d'architecture, notes de code
Designer / UX	Arbre d'information, zoning
Consultant	Cartographie processus, livrables client

2. Architecture générale

L'application suit une architecture SPA (Single Page Application) Angular à base de composants autonomes (standalone components). Elle est découpée en couches distinctes selon le pattern Feature-Core-Shared.

2.1 Vue d'ensemble

Couche	Description	Technologies
Présentation	Composants UI, templates, styles encapsulés	Angular 17, TailwindCSS, CSS custom
Services (Core)	Logique métier, état réactif, I/O	Angular Signals, RxJS, Firebase SDK
Modèles	Interfaces TypeScript typées	TypeScript 5.2
Persistance Web	Firestore (Google) / localStorage (invité)	Firebase Firestore v12
Persistance Desktop	IPC Electron → OAuth → Firebase	Electron 28, Node.js
Déploiement Web	Build Angular → CDN Netlify	Angular CLI, Netlify
Déploiement Desktop	Electron Builder → binaire portable	electron-builder 24

2.2 Diagramme de flux principal

Le flux général de l'application suit le schéma ci-dessous :

Étape	Acteur / Composant	Action
1	LoginComponent	L'utilisateur choisit Google ou Invité
2	AuthService	Firebase Auth (web popup) ou IPC Electron (PKCE)
3	AuthGuard	Vérifie isLoggedIn() → redirige vers /dashboard
4	DashboardComponent	Charge la liste via TreeMapListService
5	TreeMapListService	Abonnement Firestore (Google) ou localStorage (invité)
6	EditorShellComponent	L'utilisateur édite un TreeMap (route /editor/:id)
7	TreeService	Mutations de l'arbre via Angular Signals
8	Auto-save	Debounce 500 ms → localStorage + Firestore (si Google)
9	PrintView / Export	window.print() ou exportJson()

3. Stack technologique

Technologie	Version	Rôle
Angular	17.x	Framework frontend SPA
TypeScript	5.2.x	Langage principal
Firebase SDK	12.x	Auth Google + Firestore (cloud sync)
RxJS	7.8.x	Gestion des flux réactifs (toObservable)
TailwindCSS	3.4.x	Utilitaires CSS (config personnalisée)
Electron	28.x	Packaging desktop Windows
electron-builder	24.x	Création du binaire portable
Zone.js	0.14.x	Détection de changement Angular
Node.js	≥18	Runtime build + scripts set-env
Netlify	—	CDN + redirects SPA (netlify.toml)

3.1 Justifications des choix

Angular Signals : Réactivité fine-grained sans zone.js overhead. Remplace OnPush + async pipe pour la gestion d'état.

Firebase Firestore : Base NoSQL temps-réel, parfaitement alignée sur la structure JSON arborescente des TreeMaps.

Electron + IPC PKCE : Contournement de la restriction Chrome des popups OAuth dans les WebViews Electron. Flux sécurisé via preload.js sandboxé.

Standalone Components : Architecture Angular 17 sans NgModule : lazy-loading explicite, tree-shaking optimisé.

localStorage (mode invité) : Zéro dépendance réseau pour les utilisateurs non authentifiés ; données chiffrables côté client si besoin futur.

4. Structure du projet

L'arborescence source respecte la convention Angular feature-based avec un dossier **core** pour les services/modèles/guards partagés et un dossier **features** pour les vues.

```
treemap-editor/
  ■■■ src/
  ■ ■■■ main.ts # Bootstrap Angular
  ■ ■■■ styles.scss # Variables CSS globales (thèmes)
  ■ ■■■ environments/
  ■ ■ ■■■ environment.ts # Config Firebase (dev)
  ■ ■ ■■■ environment.prod.ts # Config Firebase (prod)
  ■ ■■■ app/
  ■ ■■■ app.component.ts # Root component ()
  ■ ■■■ app.routes.ts # Définition des routes
  ■ ■■■ core/
  ■ ■ ■■■ models/
  ■ ■ ■ ■■■ tree-node.model.ts # TreeNode, TaskItem, TreeDocument
  ■ ■ ■ ■■■ treemap.model.ts # TreeMapMeta, TreeMapDocument
  ■ ■ ■■■ services/
  ■ ■ ■ ■■■ auth.service.ts
  ■ ■ ■ ■■■ tree.service.ts
  ■ ■ ■ ■■■ treemap-list.service.ts
  ■ ■ ■ ■■■ theme.service.ts
  ■ ■ ■■■ guards/
  ■ ■ ■■■ auth.guard.ts
  ■ ■■■ features/
  ■ ■■■ auth/login/login.component.ts
  ■ ■■■ dashboard/
  ■ ■ ■■■ dashboard.component.ts
  ■ ■ ■■■ treemap-card/treemap-card.component.ts
  ■ ■■■ editor-shell/editor-shell.component.ts
  ■ ■■■ toolbar/toolbar.component.ts
  ■ ■■■ tree-view/
  ■ ■ ■■■ tree-view.component.ts
  ■ ■ ■■■ node/node.component.ts
  ■ ■■■ node-editor/node-editor.component.ts
  ■ ■■■ print-view/print-view.component.ts
  ■■■ main.js # Electron main process
  ■■■ preload.js # Electron preload (IPC bridge)
  ■■■ auth-preload.js # OAuth popup Electron
  ■■■ scripts/set-env.js # Injection des variables d'env
  ■■■ tailwind.config.js
  ■■■ angular.json
  ■■■ netlify.toml # Redirects SPA Netlify
```

5. Modèles de données

5.1 TreeNode

Unité de base de la structure arborescente. Chaque nœud est auto-contenu et référence ses enfants directement (arbre n-aire).

Champ	Type	Req.	Description
id	string (UUID)	✓	Identifiant unique généré par <code>crypto.randomUUID()</code>
title	string	✓	Libellé affiché du nœud
contentType	'text' 'image' 'tasks' 'code'	✓	Type de contenu du nœud
content	string	✓	HTML (text), URL/base64 (image), code source
children	TreeNode[]	✓	Nœuds enfants (récursif)
expanded	boolean	✓	État déplié/replié dans l'arbre
createdAt	string (ISO 8601)	✓	Horodatage de création
ticketNumber	string?	—	Référence ticket (ex : TK-123)
price	number?	—	Coût en euros
days	number?	—	Charge en jours
showPrice	boolean?	—	Visibilité du prix dans l'arbre
showDays	boolean?	—	Visibilité des jours dans l'arbre
showDescription	boolean?	—	Visibilité de la description
tasks	TaskItem[]?	—	Liste de tâches (contentType=tasks)
codeLanguage	string?	—	Langage du bloc de code
codeExpanded	boolean?	—	Bloc de code déplié par défaut

5.2 TaskItem

Champ	Type	Description
id	string (UUID)	Identifiant unique de la tâche
label	string	Texte de la tâche
completed	boolean	État coché/non coché

5.3 TreeDocument

Enveloppe d'un document TreeMap (contient la racine de l'arbre).

Champ	Type	Description
id	string (UUID)	Identifiant du document

Champ	Type	Description
name	string	Nom affiché du document
root	TreeNode	Nœud racine de l'arbre
updatedAt	string (ISO 8601)	Dernière modification

5.4 TreeMapMeta & TreeMapDocument

Métadonnées d'un projet TreeMap stockées dans la liste du dashboard.

Champ	Type	Description
id	string (UUID)	Identifiant du projet
title	string	Titre du projet
status	'en-cours' 'terminé' 'brouillon'	Statut du projet
createdAt	string (ISO 8601)	Date de création
updatedAt	string (ISO 8601)	Dernière modification
data	TreeDocument	Structure complète (TreeMapDocument uniquement)

5.5 Schéma JSON stocké dans Firestore

Chemin Firestore : `users/{uid}/treemaps/{treemapId}`. La totalité du TreeMapDocument (métadonnées + arbre) est stockée dans un seul document Firestore. La limite de taille Firestore est de 1 Mo par document.

■ ■ Pour des arbres très volumineux (> 500 nœuds), une décomposition en sous-collections sera nécessaire.

6. Services Angular

6.1 TreeService

Service central de l'éditeur. Gère l'état réactif du document courant via Angular Signals et orchestre toutes les mutations de l'arbre.

État interne

Signal / Computed	Type	Description
document (privé)	signal<TreeDocument>	Document courant (source de vérité)
doc	computed<TreeDocument>	Exposition lecture seule du document
root	computed<TreeNode>	Nœud racine du document
selectedNodeId	signal<string null>	ID du nœud sélectionné pour édition
selectedNode	computed<TreeNode null>	Nœud sélectionné (lookup O(n))

API publique

Méthode	Signature	Description
loadDocument	loadDocument(doc: TreeDocument): void	Charge un document (reset selection)
addNode	addNode(parentId: string): void	Ajoute un enfant au parent donné
removeNode	removeNode(id: string): void	Supprime un nœud (sauf racine)
updateNode	updateNode(id, updates): void	Mise à jour partielle d'un nœud
moveNode	moveNode(nodeId, targetParentId): void	Déplace un nœud (avec anti-cycle)
updateDocumentName	updateDocumentName(name: string): void	Renomme le document
getNodeTotals	getNodeTotals(node): {totalPrice, totalDays}	Calcule le prix et jours récursivement
toggleExpand	toggleExpand(id: string): void	Bascule l'état déplié/replié
toggleTask	toggleTask(nodeId, taskId): void	Coche/décoche une tâche
toggleCodeExpand	toggleCodeExpand(nodeId): void	Bascule le bloc de code
exportJson	exportJson(): void	Télécharge le document en JSON
importJson	importJson(json: string): void	Charge un document depuis JSON
resetDocument	resetDocument(): void	Réinitialise (confirmation)

Auto-save

Un **effect()** Angular écoute chaque mutation du signal *document* et émet sur un *Subject* RxJS. Un opérateur *debounceTime(500)* limite les écritures *localStorage* à une toutes les 500 ms.

6.2 AuthService

Gère l'identité de l'utilisateur. Supporte deux modes : Google OAuth et Invité. Fonctionne en mode Web (signInWithPopup) et Desktop Electron (IPC + PKCE).

Signal / Computed	Type	Description
user (privé)	signal<AuthUser null>	Utilisateur connecté ou null
loading	signal<boolean>	Indicateur de chargement OAuth
isLoggedIn	computed<boolean>	true si user() !== null
Méthode	Description	
loginWithGoogle()	Déclenche OAuth (popup web ou IPC Electron). Persiste dans localStorage.	
loginAsGuest()	Crée un utilisateur invité local (id: 'guest-{UUID}').	
logout()	Supprime l'utilisateur et appelle signOut Firebase si Google.	

■ La persistance est gérée via localStorage (clé treemap-auth-user) pour survivre aux rechargements de page sans nouvelle requête Firebase.

6.3 TreeMapListService

Orchestre la liste des projets TreeMap. Commute automatiquement entre Firestore (utilisateurs Google) et localStorage (invités) selon le signal auth.user() via toObservable().

Computed	Description
treemaps	Liste complète des TreeMapDocument
loading	true pendant la première connexion Firestore
syncError	Message d'erreur Firestore (null si ok)
total	Nombre total de TreeMaps
inProgress	Nombre de TreeMaps en statut 'en-cours'
completed	Nombre de TreeMaps en statut 'terminé'

Stratégie optimiste

Les opérations **create** et **update** appliquent d'abord la mutation localement (signal mis à jour immédiatement) puis persistent en arrière-plan (Firestore ou localStorage). L'UI reste toujours réactive.

6.4 ThemeService

Gère le thème visuel de l'application. Persiste en localStorage et applique l'attribut *data-theme* sur *document.documentElement*.

Thème	Valeur	Description
Clair	light	Thème par défaut, fond blanc
Sombre	dark	Fond foncé, texte clair
Contraste élevé	high-contrast	Noir/blanc maximal pour accessibilité

La détection initiale utilise `window.matchMedia('(prefers-color-scheme: dark)')` si aucune préférence n'est sauvegardée.

7. Composants

Composant	Chemin	Rôle
LoginComponent	app/features/auth/login/	Page de connexion (Google + Invité)
DashboardComponent	app/features/dashboard/	Tableau de bord des projets
TreemapCardComponent	app/features/dashboard/treemap-card/	Carte d'un projet dans la grille
EditorShellComponent	app/features/editor-shell/	Coquille de l'éditeur (layout principal)
ToolbarComponent	app/features/toolbar/	Barre d'outils (import/export/print/thème)
TreeViewComponent	app/features/tree-view/	Vue arborescente (liste des nœuds)
NodeComponent	app/features/tree-view/node/	Rendu récursif d'un nœud
NodeEditorComponent	app/features/node-editor/	Panneau d'édition latéral
PrintViewComponent	app/features/print-view/	Vue d'impression (visible uniquement @media print)

7.1 LoginComponent

Page d'accueil avant authentification. Propose deux actions : connexion Google (signInWithPopup ou IPC Electron) et mode invité (loginAsGuest). Affiche les erreurs Firebase traduites en français via `translateFirebaseError`.

7.2 DashboardComponent

Vue principale post-authentification. Fonctionnalités :

- Statistiques en temps réel (total / en-cours / terminés) via `computed()`.
- Barre de recherche filtrante (insensible à la casse) sur les titres.
- Grille responsive de cartes `TreemapCard`.
- Modal de création : saisie du titre → `create()` → navigation vers l'éditeur.
- Bandeau d'erreur Firestore si `syncError()` est défini.
- Bouton de déconnexion avec redirection vers `/login`.

7.3 EditorShellComponent

Conteneur de l'éditeur. Chargé via la route `/editor/:id`. Lit l'ID depuis `ActivatedRoute`, charge le `TreeMapDocument` via `TreeMapListService.getById()`, puis appelle `TreeService.loadDocument()`. Sauvegarde automatiquement à la destruction (`ngOnDestroy`) et au retour dashboard.

Sous-composants de l'éditeur

Zone	Composant	Description
Back bar	EditorShellComponent	Retour dashboard + titre + sélecteur de statut
Toolbar	ToolbarComponent	Import/Export JSON, Print, Reset, Thème
Zone gauche	TreeViewComponent	Arbre de nœuds avec expand/collapse et drag

Zone	Composant	Description
Zone droite	NodeEditorComponent	Panneau latéral d'édition (400px, glissant)
Overlay	PrintViewComponent	Contenu visible uniquement à l'impression

7.4 NodeEditorComponent

Panneau latéral d'édition. S'ouvre (width: 400px) quand selectedNodeId() est défini. Utilise un **tempNode** (copie locale) pour l'édition optimiste : les modifications ne sont propagées au TreeService qu'au clic sur « Sauvegarder ».

Type	Éditeur	Stockage
text	contenteditable div + toolbar (Bold/Italic/Underline/HTM)	HTMLstring dans content
tasks	Liste dynamique de TaskItem avec checkbox + input type="checkbox"	Task[] tasks[]
code	textarea monospace + sélecteur de langage	Chaîne brute dans content
image	Input URL + zone de drop pour upload local (FileReader + base64)	URL ou base64 dans content

7.5 PrintViewComponent

Rendu hiérarchique de l'arbre sous forme de document imprimable. Caché en affichage normal (display: none) et visible uniquement via @media print. La profondeur détermine le niveau de titre (h2 → h5). Les métadonnées (ticket, prix, jours) sont affichées si présentes.

8. Routage et guards

Route	Composant	Guard	Description
/	—	—	Redirect → /dashboard
/login	LoginComponent	guestGuard	Accessible uniquement si non connecté
/dashboard	DashboardComponent	authGuard	Accessible uniquement si connecté
/editor/:id	EditorShellComponent	authGuard	Éditeur d'un TreeMap spécifique
/**	—	—	Redirect → /dashboard

8.1 Guards

Guard	Condition d'activation	Redirection si refus
authGuard	auth.isLoggedIn() === true	/login
guestGuard	auth.isLoggedIn() === false	/dashboard

Les composants sont chargés en lazy-loading (loadComponent) pour optimiser le bundle initial.

9. Authentication

9.1 Flux Web (Navigateur)

1. Clic « Se connecter avec Gmail » → `loginWithGoogle()`.
2. Détection contexte : `isElectron() = false` → `loginWeb()`.
3. `signInWithPopup(firebaseAuth, GoogleAuthProvider)` → popup Google.
4. Firebase retourne `UserCredential` → `mapFirebaseUser()` → `_persist()`.
5. Signal `_user` mis à jour → `isLoggedIn() = true` → AuthGuard laisse passer.
6. `Router.navigate(['/dashboard'])`.

9.2 Flux Desktop (Electron)

1. Clic « Se connecter avec Gmail » → `loginWithGoogle()`.
2. Détection contexte : `isElectron() = true` (`window.ipcRenderer` présent).
3. `loginElectron()` : `ipc.invoke('google-oauth', { clientId })` → `main.js`.
4. Electron ouvre une fenêtre `BrowserWindow` dédiée (`auth-preload.js`).
5. Flux PKCE : `authorization_code` → échange token → `{ idToken, accessToken }`.
6. Retour via IPC → `signInWithCredential(firebaseAuth, GoogleAuthProvider.credential(...))`.
7. Même suite que le flux Web.

9.3 Mode Invité

`loginAsGuest()` crée un `AuthUser` local avec `provider='guest'` et un id préfixé 'guest-'. Aucun appel Firebase. Données persistées uniquement en `localStorage`. La synchronisation Firestore est désactivée.

9.4 Persistance de session

L'utilisateur est sérialisé en JSON dans `localStorage` (clé : `treemap-auth-user`). Au démarrage, `AuthService` lit cette clé dans `loadUser()`. `onAuthStateChanged` Firebase met à jour le signal si la session Firebase est toujours valide.

10. Persistance et synchronisation

Donnée	Clé localStorage	Firestore path	Mode
Utilisateur courant	treemap-auth-user	—	Toujours localStorage
Document en cours	treemap-editor-data	—	Toujours localStorage
Liste TreeMaps (invité)	treemap-list-guest	—	localStorage uniquement
Liste TreeMaps (Google)	—	users/{uid}/treemaps/{id}	Firestore (temps réel)
Thème	treemap-editor-theme	—	Toujours localStorage

10.1 Stratégie Firestore

TreeMapListService utilise **onSnapshot()** pour un abonnement temps-réel sur la sous-collection *users/{uid}/treemaps*, triée par *updatedAt* DESC. Les écritures utilisent **setDoc()** (merge total). Les suppressions utilisent **deleteDoc()**. Les erreurs Firestore sont exposées via le signal *syncError* et affichées dans le dashboard.

10.2 Désabonnement

L'unsubscribe Firestore (retour de *onSnapshot*) est stocké dans *unsub* et appelé automatiquement lors du changement d'utilisateur (déconnexion ou passage invité ↔ Google).

11. Gestion des thèmes

Les thèmes sont implémentés via des variables CSS (custom properties) définies dans *styles.scss* et appliquées à *:root* selon l'attribut *data-theme*. ThemeService change cet attribut via un *effect()* Angular.

Variable CSS	Rôle
--bg-primary	Fond principal de l'application
--bg-secondary	Fond des panneaux et cartes
--text-primary	Couleur de texte principale
--text-secondary	Couleur de texte secondaire
--accent	Couleur d'accentuation (indigo #4F46E5)
--border	Couleur des bordures
--shadow	Couleur des ombres portées
--node-hover	Survol des nœuds de l'arbre
--font-content	Police pour le contenu riche

12. Export / Import JSON

12.1 Export

`TreeService.exportJson()` sérialise le document courant (`TreeDocument`) en JSON indenté et déclenche un téléchargement via un lien `<a>` temporaire. Le nom du fichier est dérivé du nom du document (espaces → underscores).

12.2 Import

`importJson(jsonString)` parse le JSON et vérifie la présence de `root.id` comme validation minimale. En cas d'erreur de parsing, une alerte native est affichée. L'import remplace intégralement le document courant.

■■ L'import ne gère pas la migration de version : les structures JSON créées avec une version antérieure peuvent manquer des champs optionnels, ce qui est géré par les valeurs par défaut des champs optionnels (?).

13. Mode impression PDF

`PrintViewComponent` génère un rendu lisible de l'arbre entier. Il est masqué en affichage normal et visible via `@media print`. L'appel `window.print()` depuis `ToolbarComponent` déclenche la boîte de dialogue d'impression du navigateur, permettant d'enregistrer en PDF.

Profondeur	Balise HTML	Taille
Niveau 1 (racine)	h2	18pt
Niveau 2	h3	16pt
Niveau 3	h4	14pt
Niveau 4+	h5	12pt (italique)

La propriété CSS `page-break-inside: avoid` empêche la coupure au milieu d'un nœud. L'indentation est de 20px par niveau de profondeur.

14. Déploiement

14.1 Web — Netlify

La commande `npm run build:web` (`ng build --configuration production`) produit le bundle dans `dist/treemap-editor/browser/`. Netlify sert ce répertoire. Le fichier `netlify.toml` configure un redirect SPA : toutes les routes → `index.html` (status 200).

Variables d'environnement (Netlify)

Le script `scripts/set-env.js` lit les variables d'environnement système (`FIREBASE_API_KEY`, `FIREBASE_PROJECT_ID`, etc.) et génère `src/environments/environment.ts` avant le build Angular.

Variable env	Destination dans <code>environment.ts</code>
<code>FIREBASE_API_KEY</code>	<code>firebaseConfig.apiKey</code>
<code>FIREBASE_AUTH_DOMAIN</code>	<code>firebaseConfig.authDomain</code>
<code>FIREBASE_PROJECT_ID</code>	<code>firebaseConfig.projectId</code>
<code>FIREBASE_STORAGE_BUCKET</code>	<code>firebaseConfig.storageBucket</code>
<code>FIREBASE_MESSAGING_SENDER_ID</code>	<code>firebaseConfig.messagingSenderId</code>
<code>FIREBASE_APP_ID</code>	<code>firebaseConfig.appId</code>
<code>GOOGLE_CLIENT_ID</code>	<code>googleClientId</code>

14.2 Desktop — Electron

La commande `npm run electron:build` exécute `set-env.js` + `ng build` + `electron-builder`. Le résultat est un exécutable portable Windows dans le répertoire `release/`. Le processus principal (`main.js`) lance la fenêtre `BrowserWindow` avec `preload.js` qui expose `window.ipcRenderer` de manière sandboxée.

Paramètre	Valeur
<code>appId</code>	<code>com.treemap.editor</code>
<code>productName</code>	<code>TreeMapEditor</code>
<code>Output</code>	<code>release/</code>
<code>Target</code>	<code>portable (Windows)</code>
<code>Files</code>	<code>dist/treemap-editor/**, main.js, package.json</code>

15. Sécurité

15.1 Authentification

- Firebase Auth gère la validation des tokens (JWT) côté serveur.
- Les tokens Google sont échangés via le flux PKCE dans Electron (pas de client_secret exposé).
- La persistance browserLocalPersistence évite les re-authentifications inutiles.
- signOut() est appelé côté Firebase lors de la déconnexion (révocation de session).

15.2 Firestore Security Rules

La configuration Firestore doit être protégée par des règles de sécurité garantissant que chaque utilisateur ne peut accéder qu'à sa propre sous-collection :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/treemaps/{treemapId} {
      allow read, write: if request.auth != null
                           && request.auth.uid == userId;
    }
  }
}
```

15.3 Variables d'environnement

Les clés Firebase ne sont pas committées en dur. Le script set-env.js les injecte depuis les variables d'environnement système au moment du build. Le fichier environment.ts généré est gitignored en production.

15.4 XSS

Le contenu de type 'text' est rendu via [innerHTML]. Angular sanitize automatiquement le HTML via DomSanitizer. L'éditeur contenteditable utilise document.execCommand (déprécié) — une migration vers une bibliothèque d'éditeur riche (Quill, Tiptap) est recommandée.

■ Les images importées localement sont converties en base64 data URI et stockées directement dans le JSON/Firestore, ce qui peut générer des documents très volumineux pour des images haute résolution.

16. Performances

Aspect	Implémentation	Impact
Lazy loading	loadComponent() sur toutes les routes	Bundle initial réduit
Angular Signals	Réactivité fine-grained, pas de dirty-checking global	Moins de re-renders
Auto-save debounce	debounceTime(500ms) sur Subject	Max 2 écritures/s
Firestore onSnapshot	Abonnement temps-réel (delta, pas full-text)	Traffic réseau minimal
Optimistic update	Signal mis à jour avant Firestore	Latence perçue nulle
TrackBy	@for ... track tm.id dans les listes	DOM diffing optimisé

17. Contraintes et limitations connues

Taille des documents Firestore

Limite de 1 Mo par document. Les arbres avec beaucoup d'images base64 peuvent dépasser cette limite.

document.execCommand() déprécié

L'éditeur texte riche utilise une API navigateur dépréciée. Fonctionnel mais non supporté dans Safari futur.

Pas de collaboration temps-réel

Deux utilisateurs éditant le même TreeMap simultanément créeront des conflits d'écrasement (last-write-wins).

Drag & Drop non implémenté

La méthode moveNode() existe dans TreeService mais l'interface drag & drop n'est pas encore exposée dans TreeViewComponent.

Absence de tests automatisés

Aucune suite de tests unitaires ou e2e n'est configurée. Risque lors des évolutions.

Google Client ID Desktop exposé

Le googleClientId Electron est inclus dans le bundle. Acceptable pour un client desktop public mais à surveiller.

18. Évolutions futures

Priorité	Feature	Justification
Haute	Migration éditeur texte → Tiptap/Quill	Remplacer execCommand déprécié
Haute	Tests unitaires Jest + Cypress e2e	Fiabilité des évolutions
Haute	Drag & Drop UI dans TreeView	UX réorganisation nœuds
Moyenne	Sous-collections Firestore pour grands documents	Depasser la limite 1 Mo
Moyenne	Partage de TreeMap (lien lecture seule)	Collaboration async
Moyenne	Historique des versions (undo/redo)	Sécurité édition
Basse	Export Markdown / XLSX	Interopérabilité
Basse	Mode collaboratif temps-réel (CRDT)	Édition simultanée
Basse	PWA (Service Worker + mode offline)	Utilisabilité sans réseau

Document généré le 30 mai 2026 — TreeMap Editor v2.0.0 — Confidentiel